# Meta Geeks HR

---

# Proof of Concept Report

---

| Details | Information |
|---|---|
| **Project title** | **Smart Insurance AI: Health & Vehicle Fraud Detection Engine** |
| **Project Goal** | The goal of this project is to build an AI-powered insurance platform that automates claim processing and fraud detection for both health and vehicle insurance. It aims to reduce manual effort by using OCR and intelligent analysis to evaluate claims in real time. The system combines rule-based logic with AI reasoning to generate a reliable Truth Score and verdict. Ultimately, it helps insurers prevent fraud, speed up claim approvals, and improve transparency for users. |
| **Team Members** | **Yashasvi Krishna B – 2671354**<br>**Suhas Bhat B S - 2671474** |
| **Submitted by** | The Boys |
| **Submission Date** | May 4th 2026 |

# 1.INTRODUCTION

In today's digital insurance landscape, the processing and verification of claims remains one of the most operationally expensive and fraud-prone activities faced by insurers. Both the Health and Vehicle insurance sectors in India deal with enormous volumes of paper-based documents, manual verification workflows, and inconsistent fraud detection practices. These gaps result in billions of rupees lost annually to fraudulent or exaggerated claims, while genuine claimants face unnecessary delays.

That is why we built the AI-Powered Health & Vehicle Insurance Fraud Detection System — a Proof of Concept (POC) designed to transform how insurance claims are received, processed, and evaluated. Our platform integrates document intelligence, AI-driven fraud analysis, and a multi-signal Truth Score engine into a single, unified system. The goal is to provide insurers with a smart, automated, and scalable tool that makes real-time fraud decisions while reducing the manual burden on claim processing teams.

The core innovation of this system is its Truth Score Engine — a hybrid scoring mechanism that combines rule-based fraud logic with Amazon Bedrock AgentCore AI to evaluate every submitted claim across multiple fraud dimensions. For Health insurance, the system checks criteria such as diagnosis-to-amount consistency, hospital network validity, and claim mode (cashless vs. reimbursement). For Vehicle insurance, it evaluates signals like damage consistency, police report presence, claim frequency, and driver risk scores. The final Truth Score determines whether a claim is APPROVED, sent for REVIEW, or flagged as FRAUD.

The platform covers the full claim lifecycle — from user registration and document upload, through OCR-based data extraction using Amazon Textract, to AI-powered analysis and a final verdict dashboard. An integrated AI chatbot, powered by Amazon Bedrock Nova Pro, allows users and agents to query claim status, understand fraud signals, and receive professional insurance guidance through natural language conversations.

In summary, this POC demonstrates how a serverless AWS architecture combined with modern AI can make insurance fraud detection faster, more consistent, and significantly more intelligent — protecting both insurers and genuine policyholders in the process.

# 2. EXECUTIVE SUMMARY

This solution transforms the traditional, manual insurance claim verification process into a smart, AI-powered, and scalable fraud detection platform. It makes claim adjudication faster, more accurate, and data-driven by integrating document intelligence, rule-based scoring, and generative AI-based reasoning into a single workflow.

## 2.1 Overview of the Solution

We built a complete end-to-end AI Insurance Portal that handles the entire claim lifecycle — from document submission and OCR extraction to AI fraud scoring and final verdict display. The system supports two insurance verticals: Health Insurance and Vehicle Insurance, each with its own dedicated data pipeline, fraud signal set, and analysis engine.

The platform starts with a secure React-based frontend where users log in and submit claim documents. Documents are uploaded as base64-encoded files to the backend, stored in Amazon S3, and processed through Amazon Textract for OCR extraction. The extracted text is stored in Amazon DynamoDB and then forwarded to a Truth Score engine that evaluates the claim across multiple fraud dimensions using both rule-based logic and Amazon Bedrock AgentCore. The final result a Truth Score from 0 to 100 and a verdict of APPROVE, REVIEW, or FRAUD is displayed on a dedicated analysis dashboard. A conversational AI chatbot completes the experience by allowing users and agents to ask questions about claims, policies, and fraud signals.

## 2.2 Problem, Solution, User Impact, Innovation Summary

### Problem:

Insurance claim fraud in India is rampant across both the Health and Vehicle sectors. Manual verification is slow, inconsistent, and heavily dependent on the experience of individual assessors. Fraudulent claims with inflated amounts, fake diagnoses, or staged accidents frequently pass through undetected due to a lack of intelligent, automated cross-checking systems

### Solution:

We developed an AI-powered claim analysis portal that uses Amazon Textract for document OCR, Amazon Bedrock AgentCore for multi-dimensional fraud reasoning, and a hybrid Truth Score formula that combines rule-based signals with AI-generated scores to produce a final claim verdict in real time

### User Impact:

Insurance agents can process claims faster and with higher confidence. Genuine claimants experience fewer delays. Fraudulent actors are flagged before payouts are made. The platform

also enables policyholders to self-serve through the AI chatbot for claim status queries and policy guidance.

## Innovation:

The system uses Amazon Bedrock AgentCore as a harness to orchestrate multi-tool AI reasoning over claim data. It combines structured rule signals (range checks, fraud flags, driver scores) with unstructured AI narrative reasoning to produce a hybrid score — a first-of-its-kind approach for insurance fraud POC platforms on AWS.

# 3. PROBLEM STATEMENT

The insurance industry in India continues to operate with largely manual and fragmented claim verification workflows. As claim volumes grow and fraudulent tactics become more sophisticated, traditional approaches to claim adjudication are increasingly inadequate — leading to financial losses for insurers and delayed payouts for legitimate claimants.

## 3.1 Real-world Challenge

In both the Health and Vehicle insurance sectors, submitted claims arrive as physical or scanned documents. Assessors must manually read these documents, extract key data points such as claim amounts, diagnoses, hospital names, vehicle damage descriptions, and police report details, and then cross-reference them against policy records and historical patterns. This process is time-consuming, inconsistent, and highly susceptible to human error.

For Health insurance, the challenge is compounded by the variety of medical terminologies, inflated hospital bills, and the difficulty of verifying whether a claimed treatment amount is consistent with the diagnosed condition. For Vehicle insurance, fraudulent claims often involve exaggerated damage assessments, staged accidents, or missing police reports — all of which are hard to detect without automated cross-referencing and pattern analysis.

Without a unified, intelligent system, insurers cannot efficiently detect fraud signals across large claim volumes or maintain consistency in how claims are evaluated across different assessors.

## 3.2 Who is Affected

- **Insurance Companies and Claims Teams:** They bear the direct financial cost of fraudulent payouts and the operational overhead of manual verification.
- **Genuine Policyholders:** They experience longer processing times and stricter scrutiny as a result of widespread fraud in the system.
- **Insurance Agents and Assessors:** They face high workload pressure, inconsistent evaluation standards, and limited tools for intelligent fraud detection.
- **Hospital and Garage Networks:** Legitimate providers suffer reputational damage when fraudulent claims are filed in their name.

## 3.3 Evidence / Examples

- Inflated Medical Bills: Health claims frequently arrive with bill amounts significantly higher than the standard cost range for the diagnosed condition — for example, a fever treatment claim submitted for INR 85,000, well above the realistic range of INR 2,000 to INR 15,000.

- Out-of-Network Reimbursements: Claimants who submit bills from hospitals not on the insurer's approved network are inherently higher risk, yet this is rarely flagged automatically in manual processes.
- Staged Vehicle Accidents: Vehicle claims with no police report filed, combined with high repair amounts and a history of prior fraud flags, are classic indicators of staged incidents that slip through manual checks.
- Claim Frequency Abuse: Users who submit multiple claims within a short period — often across both health and vehicle verticals — are rarely identified as high-risk in the absence of centralized pattern analysis.

# 4. PROPOSED SOLUTION

The AI-Powered Health & Vehicle Insurance Fraud Detection System is an end-to-end, serverless, AI-driven platform that automates the entire claim intake, extraction, analysis, and verdict workflow. It replaces manual assessor judgment with a consistent, scalable, and intelligent evaluation engine built on AWS.

## 4.1 Solution Description

- Secure User Portal: A React.js frontend with role-based login and signup, enabling policyholders and agents to access the system from any device.
- Document Ingestion via OCR: Claim documents (medical bills, FIR copies, damage photographs) are uploaded as base64 files, stored in Amazon S3, and processed through Amazon Textract to extract raw text fields including claim amounts, diagnosis, hospital names, vehicle details, and policyholder IDs.
- Dual-Track Claim Analysis: The system maintains separate analysis pipelines for Health and Vehicle claims, each with its own DynamoDB tables, Lambda functions, and fraud signal sets tailored to the specific characteristics of each insurance type.
- Hybrid Truth Score Engine: Each claim is scored using a combination of rule-based logic (60% weight) and Amazon Bedrock AgentCore AI reasoning (40% weight). The rule engine checks diagnosis-to-amount ranges, claim mode, fraud flags, and historical patterns. The AI agent evaluates the same data across nine defined fraud dimensions and returns a structured JSON score.
- Verdict and Dashboard Display: The final Truth Score (0–100) maps to three verdicts — APPROVE (75+), REVIEW (45–74), and FRAUD (below 45) — displayed on a visual analysis dashboard with signal breakdowns.
- AI Insurance Chatbot: Amazon Bedrock Nova Pro powers a conversational assistant that can look up claim data from DynamoDB and S3, provide structured claim status summaries, and answer general insurance policy questions.

## 4.2 How It Solves the Problem

- Eliminates Manual Extraction: Amazon Textract replaces the need for assessors to manually read and transcribe claim documents, extracting text with confidence scoring.
- Standardises Fraud Evaluation: The nine-signal fraud framework applied consistently to every claim eliminates assessor bias and inconsistency.
- Enables Real-Time Decisions: The entire pipeline from upload to verdict completes in seconds, dramatically reducing claim processing time compared to manual workflows.
- Detects Multi-Signal Fraud: The AgentCore harness cross-references historical data from S3 CSV files alongside the current claim, enabling detection of patterns such as repeat offenders, outlier amounts, and policy misuse.

## 4.3 Unique Value and Innovation

- **AgentCore Harness Architecture:** The system uses Amazon Bedrock AgentCore as an orchestration layer that invokes action group tools (S3 data fetchers) before generating its fraud reasoning output — going beyond simple prompt-response AI to a tool-augmented reasoning model.
- **Dual Insurance Vertical Support:** A single frontend and backend infrastructure supports both Health and Vehicle insurance claim types with separate data models, signal sets, and analysis flows — demonstrating platform scalability from the ground up.
- **Hospital Network Verification:** The Health pipeline includes a fuzzy-match hospital network check that classifies claims as cashless (in-network) or reimbursement (out-of-network), directly impacting the fraud meter calculation.

## 4.4 Key Insight

The fundamental design principle of this system is that neither pure rule-based fraud detection nor pure AI reasoning alone is sufficient. Rule engines are fast and consistent but brittle — they miss novel fraud patterns. AI models are flexible and contextual but can be inconsistent without structure. By combining both in a 60/40 weighted hybrid score, the system achieves the reliability of deterministic rules with the reasoning depth of generative AI — producing verdicts that are both explainable and intelligent.

# 5. USE CASES

## Use Case 1: Document Upload and OCR Extraction

### Actors:

User (Policyholder or Agent), Amazon S3, Amazon Textract, AWS Lambda, Amazon DynamoDB

### Preconditions:

User is authenticated. Claim document (medical bill or vehicle claim form) is available as an image or PDF file.

### Workflow Steps:

- The user logs into the portal and selects either the Health or Vehicle insurance claim intake page.
- The user verifies a checklist of required documents before the upload button becomes active.
- The user selects a document file, which is converted to base64 in the browser.
- The frontend sends the base64-encoded file to the appropriate AWS API Gateway endpoint (healthupload or upload).
- The Lambda function (ocr_textract_health.py or ocr_textract_vehicle.py) receives the file, decodes it, and stores it in Amazon S3 under a unique timestamped key.
- Amazon Textract is invoked with the S3 object reference to perform document text detection.
- Extracted text lines and average confidence score are stored in the DynamoDB table (claim-ocr-data-health or claim-ocr-data-vehicle) with a generated Claim ID.
- The system returns the raw extracted text and Claim ID to the frontend for user review.

### Success Criteria:

Claim document is stored in S3, text is extracted with confidence above threshold, Claim ID is generated and stored in DynamoDB, and raw text is displayed to the user for verification.

## Use Case 2: Health Insurance Truth Score Analysis

### Actors:

System (AWS Lambda, Amazon Bedrock AgentCore, Amazon DynamoDB)

## Preconditions:

OCR extraction for the claim is complete. Claim ID exists in the claim-ocr-data-health DynamoDB table.

## Workflow Steps:

- The user navigates to the Health Truth Analysis page and submits the Claim ID.
- The Lambda function (truth_score_health_lambda.py) retrieves the OCR text from DynamoDB.
- The system extracts key data fields — claim amount, diagnosis, user ID, hospital name, and claim mode — using regex parsing.
- The hospital name is compared against the network hospital list to determine if the claim is cashless or reimbursement.
- The diagnosis is checked against predefined cost ranges (DISEASE_RANGES) to classify the amount as LOW, NORMAL, or HIGH.
- The Amazon Bedrock AgentCore Harness is invoked with the extracted claim data for AI-based multi-dimension fraud reasoning.
- The formula score (rule-based, 60% weight) and AI score (40% weight) are combined into the Final Truth Score.
- Adjustments are applied for HIGH range violations (-20), LOW range (-10), and reimbursement mode (-10).
- The verdict is determined: APPROVE (score >= 75), REVIEW (score 45–74), or FRAUD (score < 45).
- All signals, scores, and reasoning are stored in the fraud-results-health DynamoDB table and returned to the frontend.

## Success Criteria:

A Truth Score from 0 to 100 is generated with a clear verdict and signal breakdown displayed on the analysis dashboard.

## Use Case 3: Vehicle Insurance Truth Score Analysis

## Actors:

System (AWS Lambda, Amazon Bedrock Nova Pro, Amazon S3, Amazon DynamoDB)

## Preconditions:

OCR extraction for the vehicle claim is complete. Historical claims CSV is available in Amazon S3.

## Workflow Steps:

- The Lambda function (truth_score_vehicle_lambda.py) retrieves OCR data for the claim from DynamoDB.
- S3 action group tools fetch the user's historical claim records and system-wide average claim amounts from the vehicle_insurance_claims.csv dataset.
- The AgentCore configuration harness sends the claim data and fetched context to Amazon Bedrock Nova Pro for nine-dimension fraud reasoning: Claim Amount vs. Damage, Police Report Check, Claim Frequency, Policy Misuse, Damage Consistency, User Patterns, Outlier Detection, Duplicate Claims, and Combined Risk.
- The AI returns a structured JSON object with a truth_score and signals dictionary.
- The formula-based score and AI score are combined (60/40 weighting) to produce the Final Truth Score and verdict.
- Results are stored in fraud-results-vehicle and returned to the frontend dashboard.

## Success Criteria:

Nine fraud signals are individually scored and presented alongside the Final Truth Score and APPROVE / REVIEW / FRAUD verdict on the dashboard.

## Use Case 4: AI Chatbot — Claim and Policy Queries

## Actors:

User, AWS Lambda (chatbot.py), Amazon Bedrock Nova Pro, Amazon DynamoDB, Amazon S3

## Preconditions:

User is authenticated and has a valid session. Claim or user records exist in S3 CSV files or DynamoDB.

## Workflow Steps:

- The user opens the AI Assistant page and types a query (e.g., "Show me details for CLM-1234567890" or "What does reimbursement mean?").

- The Lambda function detects User ID (USR-) or Claim ID (CLM-) patterns in the message using regex.

- If an ID is detected, the relevant data is fetched from S3 CSV files and injected into the message as a [DATABASE RESULT] block.

- The full conversation history for the session is retrieved from DynamoDB (Chatbot_history table).
- Amazon Bedrock Nova Pro processes the message with history context and the structured system prompt to generate a professional, structured response.
- The response and updated history are saved back to DynamoDB for session continuity.
- The formatted response is returned to the user in the chat interface.

## Success Criteria:

The chatbot returns accurate, structured, and professional responses based on real claim data, maintaining session context across multiple turns.

# 6. ARCHITECTURE & DESIGN

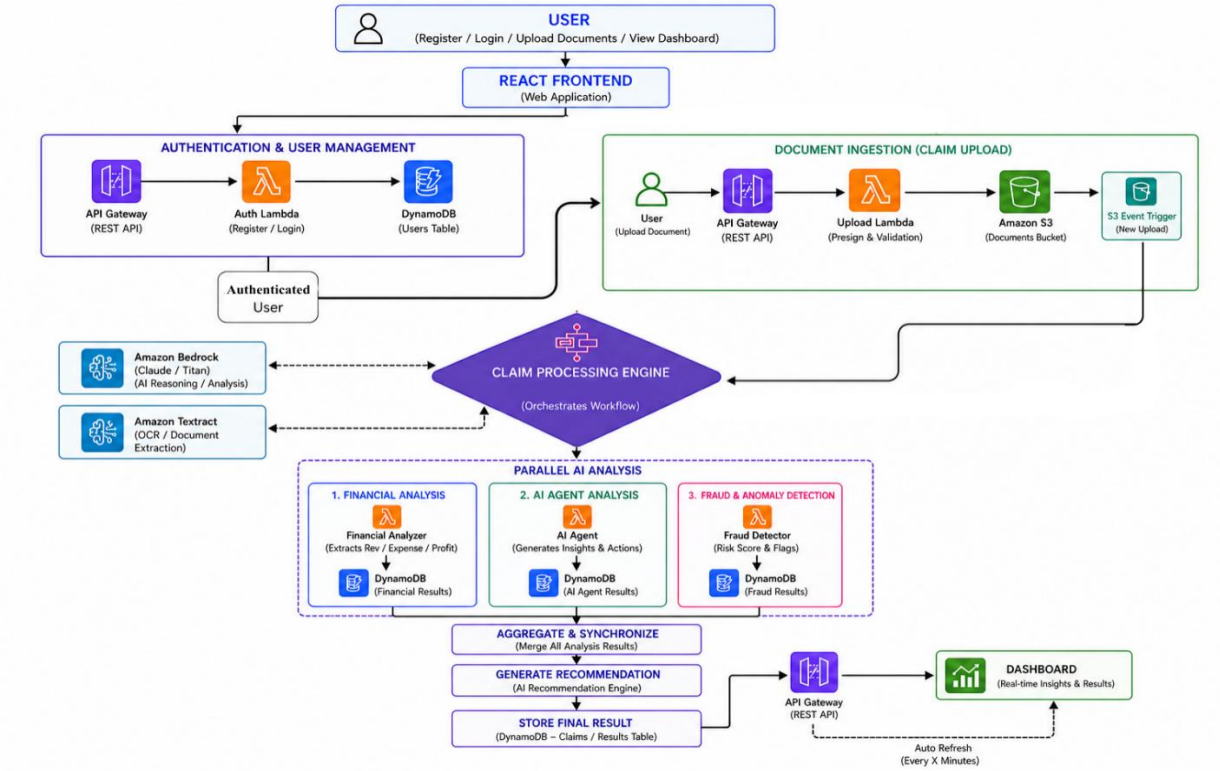
## 6.1 High-Level Architecture Workflow

The system is designed as a serverless, event-driven, and modular platform on AWS. It separates the frontend presentation layer from backend compute, storage, and AI services, ensuring that each component can scale independently without server management overhead.

The React frontend communicates exclusively through Amazon API Gateway, which routes all requests to the appropriate backend Lambda functions. User authentication is handled by dedicated Login and Signup Lambda functions backed by a DynamoDB Users table. Claim documents are uploaded via API Gateway, processed by OCR Lambda functions using Amazon Textract, and stored in both Amazon S3 (raw documents) and DynamoDB (extracted text).

The Truth Score pipeline is triggered by the frontend sending a Claim ID to the analysis endpoint. The analysis Lambda function retrieves the OCR data, runs the formula-based rule engine, invokes the Amazon Bedrock AgentCore harness for AI reasoning, combines the two scores, and stores the result back in DynamoDB. The frontend then displays the full signal breakdown and verdict in the analysis dashboard.

The AI Chatbot Lambda function maintains full session history in DynamoDB and enriches user queries with real data fetched from S3 CSV datasets before invoking Amazon Bedrock Nova Pro for response generation.



## 6.2 Technology Stack Details

### 6.2.1 Frontend Layer

| Technology | Category | Role in System |
|---|---|---|
| **React.js (Vite)** | **Core UI Framework** | Component-based SPA with Vite build tooling for fast development and optimized production bundles. |
| **Tailwind CSS** | **Styling Framework** | Utility-first CSS used to build the dark-mode glassmorphism UI theme across all portal pages. |
| **Framer Motion** | **Animation Library** | Provides smooth page transition animations and component entrance effects throughout the portal. |
| **Recharts** | **Data Visualization** | Used in dashboards to render interactive charts for health and vehicle claim trends. |
| **React Router DOM** | **Client-side Routing** | Manages dynamic routing for /health and /vehicle sub-paths and animated page transitions. |
| **Lucide React** | **Icon Library** | Provides consistent iconography across the portal UI components and dashboards. |

### 6.2.2 Backend & Compute Layer

| Technology | Category | Role in System |
|---|---|---|
| **AWS Lambda (Python 3.x)** | **Serverless Compute** | Seven Lambda functions handle authentication, OCR, truth scoring (health and vehicle), and chatbot. |
| **Amazon API Gateway** | **API Management** | Exposes RESTful endpoints to the frontend with CORS configuration and request routing. |
| **Boto3 SDK** | **AWS Integration** | Python SDK used within all Lambda functions to interact with S3, DynamoDB, Textract, and Bedrock. |

### 6.2.3 Database & Storage Layer

| Technology | Category | Role in System |
|---|---|---|
| **Amazon DynamoDB** | **NoSQL Database** | Stores user accounts, OCR claim data (health and vehicle), fraud results, and chatbot session history. |
| **Amazon S3** | **Object Storage** | Stores uploaded claim documents, historical CSV datasets (vehicle and health claims), and merged data files. |

### 6.2.4 AI & Integration Layer

| Technology | Category | Role in System |
|---|---|---|
| **Amazon Textract** | **Document OCR** | Detects and extracts text from uploaded claim documents (bills, FIRs, forms) with confidence scoring. |
| **Amazon Bedrock AgentCore** | **AI Orchestration** | Harness framework used for health claim fraud reasoning — orchestrates tool calls and structured JSON output. |
| **Amazon Bedrock Nova Pro** | **Generative AI Model** | Powers both the vehicle fraud reasoning engine and the AI chatbot for claim queries and policy guidance. |
| **Amazon Bedrock Runtime** | **Model Invocation** | Direct API used for Nova Pro model calls in the vehicle truth score and chatbot Lambda functions. |

# 7. FEATURES & CAPABILITIES

| Feature | Category | Description |
|---|---|---|
| **Dual Insurance Portal** | **Core Feature** | Single platform supporting both Health and Vehicle insurance claim workflows with separate analysis pipelines. |
| **OCR-Based Document Extraction** | **Data Intelligence** | Amazon Textract extracts claim details from uploaded images and PDFs with average confidence scoring. |
| **Health Truth Score Engine** | **Fraud Detection** | Hybrid rule + AI scoring evaluating diagnosis-amount ranges, hospital network status, and claim mode. |
| **Vehicle Truth Score Engine** | **Fraud Detection** | Nine-signal AI fraud analysis covering damage consistency, police reports, claim frequency, and user history. |
| **AgentCore Harness** | **AI Innovation** | Orchestrates multi-tool AI reasoning using Bedrock AgentCore for structured JSON fraud signal output. |
| **Fraud Meter Indicator** | **Risk Visualization** | Dynamic fraud risk meter displayed alongside Truth Score for immediate visual risk assessment. |
| **Verdict System** | **Decision Engine** | Three-tier verdict output: APPROVE, REVIEW, and FRAUD based on final Truth Score thresholds. |
| **AI Insurance Chatbot** | **User Experience** | Session-aware conversational assistant powered by Nova Pro that can look up real claim and user data. |
| **Hospital Network Check** | **Health Specific** | Fuzzy-match algorithm identifies in-network hospitals, classifying claims as cashless or reimbursement. |

| | | |
|---|---|---|
| **Signal Breakdown Dashboard** | **Transparency** | Each fraud signal is individually displayed with a score and reasoning for full adjudication transparency. |
| **Secure Authentication** | **Security** | SHA-256 password hashing, Gmail-only registration, and DynamoDB-backed user management. |
| **Toast Notification System** | **UI/UX** | Global real-time toast notifications for upload success, errors, and session events across the portal. |
| **Responsive Dark-Mode UI** | **UI/UX** | Glassmorphism design theme with full mobile responsiveness, collapsible sidebar, and animated transitions. |

# 8. INSTALLATION GUIDE

## 8.1 System Requirements

| Requirement | Specification |
|---|---|
| **Operating System** | Windows 10/11, macOS 12+, or Ubuntu 20.04+ |
| **Node.js** | v18.x or above (required for Vite / React build) |
| **Python** | 3.10 or above (for Lambda function development and local testing) |
| **AWS CLI** | v2.x configured with IAM credentials (AdministratorAccess for POC setup) |
| **AWS Account** | Active AWS account with access to Lambda, API Gateway, S3, DynamoDB, Textract, and Bedrock |
| **Browser** | Chrome 110+, Firefox 115+, or Safari 16+ for frontend access |
| **Git** | Any modern version for code checkout |

## 8.2 Backend Setup (AWS Lambda)

- Log into the AWS Console and navigate to the AWS Lambda service in the us-east-1 region.
- Create seven Lambda functions: login_lambda, signup_lambda, ocr_textract_health, ocr_textract_vehicle, truth_score_health_lambda, truth_score_vehicle_lambda, and chatbot.
- For each function, set the runtime to Python 3.12 and upload the corresponding .py file from the backend/lambda_functions/ directory.
- Attach the following IAM permissions to the Lambda execution role: AmazonS3FullAccess, AmazonDynamoDBFullAccess, AmazonTextractFullAccess, AmazonBedrockFullAccess.
- Configure Amazon API Gateway (REST API) with POST routes: /login, /signup, /upload, /healthupload, /analyse, /healthanalyse, /chat. Enable CORS on all routes. Deploy the API to a stage (e.g., prod).
- Set the environment variable BUCKET_NAME to ai-insurance-data-suhas (or your chosen bucket name) on the Lambda functions that require S3 access.
- For the health truth score Lambda, set the HARNESS_ID environment variable to the AgentCore Harness ID from your Amazon Bedrock AgentCore configuration.

## 8.3 Frontend Setup

- Clone or extract the project repository to your local machine.

- Navigate to the frontend/ directory and create a .env file based on the provided frontend/.env template.
- Set VITE_AWS_API_URL to the Invoke URL of your deployed API Gateway stage (e.g., https://<api-id>.execute-api.us-east-1.amazonaws.com/prod).
- Run **npm install** to install all frontend dependencies (React, Vite, Tailwind CSS, Framer Motion, Recharts, Lucide React).
- Run **npm run dev** to start the local development server, or **npm run build** followed by **npm run** preview for a production build preview.
- For production deployment, upload the generated **dist/** folder to an S3 bucket configured for static website hosting or deploy to AWS Amplify / Onrender.

## 8.4 Database & Storage Setup

- Create the following DynamoDB tables in us-east-1 with On-Demand billing mode:
    - **Users_Login_Signup** — Partition Key: user_id (String)
    - **claim-ocr-data-health** — Partition Key: claim_id (String)
    - **claim-ocr-data-vehicle** — Partition Key: claim_id (String)
    - **fraud-results-health** — Partition Key: claim_id (String)
    - **fraud-results-vehicle** — Partition Key: claim_id (String)
    - **Chatbot_history** — Partition Key: session_id (String)

- Create an S3 bucket named **ai-insurance-data-suhas** (or your preferred name) in us-east-1. Upload the following CSV reference files to the bucket root: vehicle_insurance_claims.csv, health_insurance_claims.csv, users_master.csv, and claims_history.csv.
- Ensure the Lambda execution role has s3:GetObject and s3:PutObject permissions on this bucket.

## 8.5 AI Model Setup

- Navigate to the Amazon Bedrock console in us-east-1 and request access to the Amazon Nova Pro model (model ID: us.amazon.nova-pro-v1:0) under Model Access.
- For Health insurance analysis, set up an Amazon Bedrock AgentCore Harness. Create a new harness in the AgentCore console, configure the system prompt and action groups for S3 data fetching, and note the Harness ID to use as the HARNESS_ID environment variable.
- Verify that the Lambda execution role has bedrock:InvokeModel and bedrock-agent-runtime:InvokeAgent permissions.

# 9. USER MANUAL

## 9.1 Login & Signup Steps

- Open the portal URL in your browser. You will be directed to the login/signup page.
- To create a new account, click Sign Up. Enter your full name, a valid Gmail address (only @gmail.com accounts are accepted), and a password of at least 6 characters. Click Sign Up. A success toast will confirm account creation.
- To log in, enter your registered Gmail and password and click Login. A welcome toast will appear and you will be redirected to the main dashboard.

## 9.2 Action Steps — Selecting Insurance Type

- The main dashboard presents two entry cards: Health Insurance and Vehicle Insurance.
- Click on the relevant card to enter the corresponding insurance portal. The URL will update to /health/dashboard or /vehicle/dashboard.
- The left sidebar navigation provides access to Dashboard, Submit Claim, Truth Analysis, AI Assistant, and Architecture pages specific to the selected insurance type.

## 9.3 Triggering the Workflow — Claim Submission

- Navigate to the Submit Claim page from the sidebar.
- Review and check all items in the document readiness checklist. The upload button will only activate once all checklist items are confirmed.
- Click the upload area and select the claim document file (supported: PNG, JPG, PDF). The file is automatically encoded and sent to the OCR endpoint.
- Wait for the processing indicator to complete. The system will display the extracted OCR text and the generated Claim ID (format: CLM-<timestamp>).
- Note the Claim ID shown on screen — it is required for the Truth Analysis step.

## 9.4 Viewing Outputs — Truth Analysis Dashboard

- Navigate to the Truth Analysis page from the sidebar.
- The page will automatically use the Claim ID from the previous submission (passed via React Router state). If accessing directly, the system uses a default test Claim ID for demo purposes.
- A loading animation plays while the analysis Lambda function processes the claim. The result screen displays: Final Truth Score (0–100), Verdict badge (APPROVE / REVIEW / FRAUD), Fraud Meter gauge, individual signal cards with scores and reasoning, and the AI-generated narrative reasoning from Bedrock.

## 9.5 Troubleshooting

| Issue | Resolution |
|---|---|
| **Upload fails immediately** | Verify all checklist items are checked. Check browser console for CORS errors — confirm API Gateway CORS headers are configured correctly. |
| **OCR returns empty text** | Ensure the uploaded document is a clear, high-resolution image or PDF. Low-quality scans may fail Textract extraction. |
| **Truth Score shows no signals** | Confirm the Claim ID exists in the DynamoDB OCR table. Check the Lambda function CloudWatch logs for AgentCore invocation errors. |
| **Chatbot returns generic responses** | Ensure your S3 bucket contains the required CSV files. Verify the Lambda has S3 read permissions. Include a valid USR- or CLM- ID in your message for data-backed responses. |
| **Login fails with valid credentials** | Passwords are SHA-256 hashed. If the account was created through a different method, it may not match. Re-register if needed. |

# 10. DEMO INSTRUCTIONS

## 10.1 Demo Assets

- Frontend portal running on localhost:5173 (dev) or deployed on S3/Amplify URL.
- Sample health claim document image (e.g., a mock hospital bill with diagnosis, amount, and hospital name fields).
- Sample vehicle claim document image (e.g., a mock FIR/damage assessment form).
- AWS Console access to demonstrate Lambda executions and DynamoDB records (optional for technical audience).
- CSV reference data files uploaded to S3: vehicle_insurance_claims.csv and health_insurance_claims.csv with at least 50 records each.

## 10.2 Test Credentials

| Credential | Value |
|---|---|
| **Test Email** | admin@gmail.com (or register a new account during demo) |
| **Test Password** | admin123 (minimum 6 characters required) |
| **Sample Claim ID (Health)** | CLM-1777281763 (pre-loaded in DynamoDB for demo) |
| **Sample User ID** | USR-001 (available in S3 CSV data for chatbot lookups) |

## 10.3 Execution Steps — Demo Flow

- Open the portal and walk through the login screen. Demonstrate form validation (e.g., try a non-Gmail email to show the error message).
- Log in with valid test credentials. Show the main dashboard with the two insurance type cards.
- Select Health Insurance. Navigate to Submit Claim. Check all checklist items and upload the sample health claim document.
- Show the OCR result screen with extracted text and the generated Claim ID displayed on screen.
- Navigate to Truth Analysis. Allow the loading animation to complete. Walk through the Truth Score display, explaining the verdict, fraud meter, and individual signal cards.
- Switch to Vehicle Insurance. Repeat the claim submission and analysis flow with the vehicle sample document.

- Open the AI Assistant. Type a query referencing the test Claim ID (e.g., "Tell me about CLM-1777281763"). Show the structured AI response with bullet-point data presentation.
- Optionally, show the AWS Console — Lambda function logs, DynamoDB tables with stored claim data, and S3 bucket with uploaded documents — to demonstrate the real backend infrastructure.

# 11. INNOVATION & IMPACT

## 11.1 Business Impact

- **Fraud Loss Reduction:** By automatically flagging high-risk claims with FRAUD or REVIEW verdicts before payout, the system directly reduces fraudulent payouts. Even a 10% improvement in fraud detection at scale translates to significant financial savings for insurers.
- **Operational Efficiency:** Automating document extraction and fraud evaluation eliminates the bottleneck of manual claim assessment, enabling insurers to process significantly higher claim volumes with the same or fewer resources.
- **Standardised Adjudication:** Consistent nine-signal evaluation across every claim eliminates assessor variability, reducing the risk of both under-detection (missed fraud) and over-detection (wrongful rejection of legitimate claims).

## 11.2 User Impact

- **Faster Claim Decisions:** Genuine claimants benefit from dramatically faster turnaround times as the system's analysis completes in seconds rather than days.
- **Transparency**: The signal breakdown and AI reasoning narrative give policyholders and agents a clear, explainable basis for every decision — moving away from opaque "black box" rejections.
- **Self-Service Intelligence:** The AI chatbot empowers policyholders to query their own claim status and understand insurance concepts without waiting for agent availability.

## 11.3 Technical Innovation

- **AgentCore Harness Pattern:** Using Amazon Bedrock AgentCore as a multi-tool reasoning orchestrator for insurance fraud is a novel architectural approach that goes beyond standard prompt engineering — enabling the AI to fetch and reason over real external data before producing its verdict.
- **Hybrid Scoring Formula:** The 60/40 weighted combination of deterministic rule scores and probabilistic AI scores creates a scoring mechanism that is both auditable and adaptive — a design pattern with broad applicability across financial risk domains.
- **Dual-Vertical Insurance Platform:** Building a single serverless platform that supports two structurally different insurance claim types (medical and vehicle) with shared infrastructure but specialised analysis pipelines demonstrates scalable multi-domain AI application design.

## 11.4 Scalability Benefits

- **Fully Serverless:** All compute runs on AWS Lambda, which scales automatically to handle any volume of concurrent claim submissions without pre-provisioning.
- **Decoupled Storage:** Separating raw document storage (S3) from structured claim data (DynamoDB) allows each layer to scale independently.
- **Multi-Region Ready**: The architecture can be extended to multi-region AWS deployments to serve national or international insurance operations with low latency.

# 12. TECHNICAL DETAILS

## 12.1 API Specifications

| Endpoint | Description & Request Body | Response |
|---|---|---|
| **POST /login** | Authenticate existing user. Body: { email, password } | 200 OK with email and name; 400/401/404 for validation or auth failures. |
| **POST /signup** | Register new user. Body: { email, password, name } | 200 OK on success; 400 if user exists or validation fails. |
| **POST /healthupload** | Upload and OCR-extract health claim document. Body: { file_base64, file_name, claim_type } | 200 OK with claim_id, raw_text, and confidence_pct. |
| **POST /upload** | Upload and OCR-extract vehicle claim document. Body: { file_base64, file_name, claim_type } | 200 OK with claim_id, raw_text, and confidence_pct. |
| **POST /healthanalyse** | Run health insurance Truth Score analysis. Body: { claim_id } | 200 OK with truth_score, verdict, signals, and reasoning. |
| **POST /analyse** | Run vehicle insurance Truth Score analysis. Body: { claim_id } | 200 OK with truth_score, verdict, nine signals, and reasoning. |
| **POST /chat** | Send message to AI chatbot. Body: { message, session_id } | 200 OK with AI-generated structured response text. |

## 12.2 Data Models

**DynamoDB: claim-ocr-data-health / claim-ocr-data-vehicle**

| Field | Description |
|---|---|
| **claim_id** | String (PK) — Format: CLM-<unix_timestamp> |
| **s3_key** | String — Path to the uploaded document in S3 |

| | |
|---|---|
| **file_name** | String — Original file name of the uploaded document |
| **ocr_text** | String — Full raw text extracted by Amazon Textract |
| **line_count** | Number — Count of extracted text lines |
| **confidence_pct** | Decimal — Average Textract confidence score (0–100) |
| **claim_type** | String — "health" or "vehicle" |
| **timestamp** | String — UTC timestamp of extraction |

**DynamoDB: fraud-results-health / fraud-results-vehicle**

| Field | Description |
|---|---|
| **claim_id** | String (PK) — Matches OCR table claim_id |
| **truth_score** | Number — Final hybrid score (0–100) |
| **formula_score** | Number — Rule-based component score |
| **ai_score** | Number — Bedrock AI component score |
| **verdict** | String — "APPROVE", "REVIEW", or "FRAUD" |
| **signals** | Map — Dictionary of individual fraud signal scores and reasons |

| | |
|---|---|
| **reasoning** | String — AI-generated narrative explanation of the verdict |
| **fraud_meter** | Number — Fraud risk meter value (0–100) for UI display |
| **range_status** | String — "LOW", "NORMAL", "HIGH" for health claim amount ranges |
| **claim_mode** | String — "cashless" or "reimbursement" (health claims) |

## 12.3 Security & Encryption

- Passwords are SHA-256 hashed before storage in DynamoDB. Plain-text passwords are never stored or logged.
- Only Gmail accounts (@gmail.com) are accepted for registration, validated server-side using regex in both Lambda functions.
- All API Gateway endpoints are configured with CORS headers allowing cross-origin requests from the frontend domain.
- S3 buckets are not publicly accessible. All document access is through IAM-authenticated Lambda execution roles.
- Amazon Bedrock model invocations are secured through AWS IAM role-based access attached to Lambda execution roles.

## 12.4 Performance & Scalability

- OCR Extraction: Amazon Textract processes a standard A4 document image in approximately 2–5 seconds, depending on document complexity.
- Truth Score Analysis: The full Lambda execution — including DynamoDB read, rule-based scoring, Bedrock invocation, and DynamoDB write — typically completes in 5–15 seconds for health claims and 8–20 seconds for vehicle claims with AgentCore tool invocations.
- Concurrent Requests: AWS Lambda automatically scales to handle concurrent claim submissions. The system can process thousands of claims in parallel without configuration changes.
- DynamoDB Performance: All tables use On-Demand billing mode with single-digit millisecond read/write latency at any scale.

# 13. LIMITATIONS

## 13.1 Known Issues

- The vehicle TruthAnalysisVehicle.jsx frontend component is currently commented out as a placeholder — a live integration with the /analyse endpoint needs to be wired into the active component version for the vehicle analysis dashboard to display real signal data.

- The hospital_network.py Lambda function is also in a commented-out state in the current codebase and relies on a hospital-network DynamoDB table that must be populated with hospital records before the network verification logic can be activated.

- The ClaimSubmission.jsx frontend component is commented out — the production version with real API calls needs to be uncommented and validated for both health and vehicle upload flows.

- Error handling in the frontend for network timeouts or Lambda cold starts is minimal — extended loading states may appear as blank screens rather than informative error messages.

## 13.2 Mock Data

- The S3 CSV datasets (vehicle_insurance_claims.csv and health_insurance_claims.csv) used for user history and system averages in the fraud analysis are synthetic datasets for POC demonstration purposes.

- The default fallback Claim ID (CLM-1777281763) hardcoded in TruthAnalysisHealth.jsx is a pre-seeded test record for demo purposes and may not reflect realistic claim data in all deployment environments.

- The DISEASE_RANGES dictionary in the health Lambda defines static cost ranges for six conditions. In a production system, these ranges would be dynamically sourced from an actuarial database.

## 13.3 Time Constraints

- Due to POC timeline constraints, the Architecture.jsx page renders a static architecture diagram rather than a dynamically generated infrastructure map.

- Multi-factor authentication and JWT-based session tokens have not been implemented — the current auth system relies on DynamoDB password hash comparison without token-based session management.

- The chatbot does not support file attachments or image inputs in the current POC build — only text-based queries and ID-referenced data lookups are supported.

# 14. FUTURE ENHANCEMENTS

## 14.1 Additional Features

- JWT-based authentication with token refresh, session expiry, and multi-factor authentication using Amazon Cognito.
- Agent and Policyholder role-based access control — agents can view all claims, policyholders can only view their own.
- Automated claim status notifications using Amazon SNS — policyholders receive email or SMS alerts when their claim verdict is updated.
- Feedback loop for assessors — the ability to override AI verdicts and feed those corrections back as training signal to continuously improve fraud detection accuracy.
- PDF report generation — one-click export of the full Truth Score analysis and signal breakdown as a PDF document for regulatory record-keeping.

## 14.2 Integrations

- Integration with national hospital network APIs (e.g., NHA Ayushman Bharat hospital registry) for real-time cashless eligibility verification.
- Integration with vehicle registration and driver history APIs (e.g., Vahan portal) to validate vehicle details and driver records submitted in claims.
- Integration with insurance bureau blacklist databases to cross-check submitting users against known fraudulent actors.
- Integration with Amazon Comprehend Medical for structured extraction of medical entities (conditions, medications, procedures) from health claim documents.

## 14.3 Production Readiness Steps

- Replace all hardcoded Lambda environment values (bucket names, table names, model IDs) with AWS Systems Manager Parameter Store or Secrets Manager references.
- Implement AWS WAF on the API Gateway to protect against DDoS attacks, SQL injection attempts, and credential stuffing on the authentication endpoints.
- Migrate the frontend to a CloudFront + S3 static hosting deployment with HTTPS enforcement and custom domain configuration.

- Set up CloudWatch dashboards and alarms for Lambda error rates, Textract failure rates, and Bedrock invocation latency to enable proactive production monitoring.
- Commission a third-party security audit of the authentication flow, S3 access policies, and DynamoDB data at-rest encryption configuration before production launch.

# 15. CONCLUSION

## 15.1 Summary of Value

This Proof of Concept demonstrates that a fully functional, AI-powered insurance fraud detection platform can be built on AWS serverless infrastructure within a focused development sprint. The system successfully delivers end-to-end claim processing — from secure user authentication and document upload through OCR extraction and multi-dimensional AI fraud analysis to a final, transparent verdict — across both Health and Vehicle insurance verticals.

The hybrid Truth Score engine represents the core technical contribution of this POC: a combination of deterministic rule-based signals and Amazon Bedrock AgentCore AI reasoning that produces verdicts which are simultaneously explainable, consistent, and intelligent. This scoring approach is directly applicable to production insurance environments and can be extended to additional insurance verticals, claim types, or risk domains with minimal architectural changes.

## 15.2 Final Pitch Message

Insurance fraud is not a data problem — it is an intelligence problem.

The AI-Powered Health & Vehicle Insurance Fraud Detection System proves that real-time, explainable, and scalable fraud intelligence is achievable today on AWS. By combining the consistency of rule-based scoring with the reasoning depth of Amazon Bedrock AI, we have built a system that does not just detect fraud — it understands it.

This POC is ready to be extended into a production-grade fraud detection platform that protects insurers, accelerates genuine claims, and brings intelligent transparency to every adjudication decision.

# 16. APPENDIX

## API Reference & Documentation Links:

## AWS Services Documentation

| Service | Documentation URL |
|---|---|
| **Amazon Textract** | https://docs.aws.amazon.com/textract/latest/dg/what-is.html |
| **Amazon Bedrock** | https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html |
| **Amazon Bedrock AgentCore** | https://docs.aws.amazon.com/bedrock/latest/userguide/agent-core.html |
| **Amazon Nova Pro Model** | https://docs.aws.amazon.com/bedrock/latest/userguide/titan-models.html |
| **AWS Lambda (Python)** | https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html |
| **Amazon API Gateway** | https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html |
| **Amazon DynamoDB** | https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html |
| **Amazon S3** | https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html |
| **Boto3 SDK Reference** | https://boto3.amazonaws.com/v1/documentation/api/latest/index.html |

## Frontend Libraries Documentation

| Library | Documentation URL |
|---|---|
| **React.js** | https://react.dev/ |
| **Vite Build Tool** | https://vitejs.dev/guide/ |
| **Tailwind CSS** | https://tailwindcss.com/docs/ |
| **Framer Motion** | https://www.framer.com/motion/ |
| **Recharts** | https://recharts.org/en-US/api |
| **React Router DOM** | https://reactrouter.com/en/main |
| **Lucide React Icons** | https://lucide.dev/guide/packages/lucide-react |

## Fraud Detection & AI Reference

| Resource | URL |
|---|---|
| **IRDAI Insurance Fraud Guidelines (India)** | https://www.irdai.gov.in/ |
| **Amazon Bedrock Model IDs Reference** | https://docs.aws.amazon.com/bedrock/latest/userguide/model-ids.html |
| **Amazon Comprehend Medical** | https://docs.aws.amazon.com/comprehend/latest/dg/comprehend-med.html |
| **AWS Well-Architected Framework** | https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html |